Architectural Principles

Purpose

The following principles define the architectural foundation of Shio Runtime.

Every Runtime, Adapter, feature and future architectural decision must comply with these principles.

They exist to preserve long-term architectural consistency as the project evolves.

Whenever implementation and architectural principles appear to conflict, the architecture should be reconsidered before implementation proceeds.

Principle 1 — Responsibilities Before Technologies

> **Shio Runtime is designed around responsibilities, not technologies.**

Architecture should never be organized around libraries, providers or frameworks.

Instead, it should be organized around responsibilities.

Examples:

- Speech Runtime
- Avatar Runtime
- Action Runtime
- Experience Layer
- Memory Layer

are architectural concepts.

Examples:

- Groq
- OpenAI
- Three.js
- Electron
- SQLite
- Python Speech

are implementation technologies.

Technologies may change.

Responsibilities should remain stable.

Principle 2 — Runtime Executes, LLM Decides

> **LLMs propose. Runtime decides. Runtime executes.**

The language model is responsible for reasoning, conversation and proposing intentions.

The Runtime is responsible for:

- validation;
- coordination;
- execution;
- safety;
- state management.

The language model never directly controls the application.

The Runtime always owns the final decision.

Principle 3 — One Runtime, One Responsibility

> **Every Runtime owns exactly one responsibility.**

Each Runtime should solve one clearly defined problem.

Responsibilities must never overlap.

When a Runtime begins accumulating unrelated logic, architectural boundaries should be reconsidered before further development continues.

Simple Runtime systems scale better than large monolithic controllers.

Principle 4 — Everything External Is an Adapter

> **Everything outside the Runtime is an Adapter.**

External technologies are never considered part of the Runtime.

Examples include:

- LLM providers;
- speech providers;
- streaming platforms;

- operating systems;
- storage engines;
- rendering libraries.

Runtime systems communicate with architectural concepts.

Adapters communicate with technologies.

This separation allows implementation technologies to evolve independently from the Runtime.

Principle 5 — Runtime Owns Reality

> **Nothing changes the outside world except Action Runtime.**

Action Runtime is the sole architectural authority permitted to produce observable side effects outside the Runtime.

Every external action must pass through:

- Tool Registry;
- Permission Guard;
- Action Runtime;
- External Adapter.

No other Runtime is allowed to bypass this boundary.

This principle guarantees:

- safety;
- consistency;
- auditability;
- platform independence.

Principle 6 — Experience Is As Important As Execution

> **Execution changes the world. Experience changes the character.**

Completing an action is not the end of an interaction.

Every meaningful event may become a character experience.

Experience Layer determines:

- emotional reactions;

- post-action dialogue;
- expressive behavior;
- intentional silence.

Execution and experience remain separate architectural responsibilities.

Principle 7 — Preserve the Illusion

> **Every Runtime exists to preserve the illusion of a living character.**

Every architectural decision should strengthen the user's perception that they are interacting with a coherent digital character.

Examples include:

- breathing;
- blinking;
- emotional continuity;
- memory;
- natural timing;
- expressive reactions.

Technical sophistication alone is never the objective.

Believability is.

Principle 8 — The Runtime Exists To Disappear

> **The Runtime should never be visible. Only Shio should be visible.**

Users should never perceive Runtime systems.

They should never think:

> "The animation controller reacted."

They should think:

> "Shio smiled."

They should never think:

> "The moderation system executed."

They should think:

"Shio banned the spammer."

The better the Runtime becomes, the less visible it is.

Implementation disappears.

The character remains.

Principle 9 — Character Before Features

The character is the product. Features exist to support the character.

New capabilities should strengthen the illusion of interacting with Shio.

Features should never become architectural goals by themselves.

Whenever priorities conflict:

- preserving character consistency;
- preserving personality;
- preserving continuity;

should take precedence over adding isolated functionality.

Principle 10 — Composition Over Complexity

Complex behavior should emerge from simple cooperating Runtime systems.

Whenever possible, new capabilities should be created by composing existing Runtime systems instead of introducing large monolithic controllers.

Examples include combining:

- Speech Runtime;
- Experience Layer;
- Memory Layer;
- Action Runtime;
- Character State;

rather than creating feature-specific orchestration systems.

Composition produces architectures that are easier to extend, test and understand.

Principle 11 — Long-Term Evolution Over Short-Term Convenience

Architecture should optimize for years of evolution, not today's implementation.

Implementation shortcuts that compromise Runtime boundaries should be avoided.

Every architectural decision should remain valid after:

- replacing technologies;
- introducing new Runtime systems;
- adding new external platforms;
- extending character capabilities.

Architecture is expected to outlive every individual technology used to implement it.

Summary

Together, these principles define the architectural identity of Shio Runtime.

Every Runtime, every Adapter and every future feature should reinforce these ideas rather than weaken them.

When uncertainty exists, the following questions should always be asked:

- Which Runtime owns this responsibility?
- Does this preserve the illusion of the character?
- Does this introduce unnecessary coupling?
- Can this technology be replaced without changing the architecture?
- Does this strengthen Shio rather than expose the Runtime?

If these questions can be answered confidently, the implementation is likely aligned with the architecture.

If they cannot, the architecture should be reconsidered before implementation begins.